

Memoria P 5 - ADSOF

Alejandro Seguido y Rubén García

Apartado 1.

La clase **Feature** se ha diseñado para almacenar secuencias de objetos de cualquier tipo, con la restricción de que estos sean comparables entre sí. Esto se implementa mediante la declaración genérica `Feature<T extends Comparable<T>>`. Esta restricción garantiza que se puedan determinar los valores máximo y mínimo.

- **Extensión de `ArrayList<T>`:** Se ha optado por que `Feature` herede de `ArrayList<T>` para garantizar que sea compatible con las listas estándar de Java.
- **Operaciones de Agregación:** Para la obtención de los valores extremos (min y max), se ha implementado un enfoque iterativo que recorre la colección completa.

Para desacoplar la extracción de datos de la estructura de almacenamiento, se ha definido la interfaz **`IFeaturizer<T>`**. Esta interfaz abstrae la lógica necesaria para extraer características específicas de un objeto genérico `T`.

Se compone de dos métodos fundamentales: uno para listar los nombres de las características de interés y otro para extraer el valor de una característica específica de un objeto dado.

* Retorno Genérico: El método de extracción devuelve un objeto `Comparable<?>`. Así el retorno es el más general posible y cumpliendo con el único requisito requerido por la clase `Feature` (los valores extraídos deben ser comparables).

La clase **`Dataset`** actúa como el contenedor principal, que extrae características usando una clase que implemente `IFeaturizer<T>`.

- Características: se utiliza un `LinkedHashMap<String, Feature<?>>` para mapear el nombre de cada característica con su correspondiente lista `Feature`. El uso de `LinkedHashMap` es crucial ya que preserva el orden de inserción de las características. El comodín `'?'` en `Feature<?>` se debe a que no sabemos de que tipo pueden ser las características, ni siquiera si son todas del mismo tipo.
- Datos: Los objetos originales se almacenan en un `LinkedHashSet<T>`. Esto permite mantener el orden de inserción de los elementos procesados.

Inserción de Datos (`addAll`): Este método procesa un array de objetos, utilizando el `IFeaturizer` para extraer los valores de interés e insertarlos en sus respectivas columnas (`Feature`) dentro del mapa.

Recuperación de características: Para acceder a una característica específica y evitar problemas de casteo manual en el código cliente, se provee el método:

```
public <R extends Comparable<R>> Feature<R> feature(String featureName)
```

Este diseño utiliza genéricos a nivel de método para realizar un casting seguro al tipo esperado R.

Eliminación de Duplicados: dos objetos se consideran idénticos si y sólo si todas sus características de interés son equivalentes. Utiliza un HashSet auxiliar para identificar combinaciones de valores únicas. Finalmente, si se detectan duplicados, el dataset se reconstruye.

Apartado 2.

Para garantizar un diseño modular, extensible y de fácil mantenimiento, la arquitectura del árbol de decisión se ha dividido en tres componentes fundamentales: `DecisionTree`, `Node` y `Rama`.

1. `DecisionTree<T>`

La clase genérica `DecisionTree<T>` actúa como el contenedor y orquestador principal de la estructura jerárquica. Las decisiones de diseño clave en este componente incluyen:

- **Acceso Optimizado:** Se implementa un diccionario (`Map<String, Node<T>>`) que asocia la etiqueta de cada nodo con su instancia en memoria. Esto proporciona una complejidad de acceso de tiempo constante ($O(1)$), lo que agiliza drásticamente la construcción y consulta del grafo.
- **Mecanismo Anti-Ciclos:** Para preservar la naturaleza de un árbol (un Grafo Acíclico Dirigido donde cada nodo tiene exactamente un padre), se emplea un conjunto `Set<Node<T>>` denominado `nodosVisitados`. Este registro actúa como un escudo protector: si al construir el árbol se intenta establecer como destino un nodo que ya existe en el conjunto, la operación es descartada. Esto impide colisiones estructurales o bucles infinitos.
- **Lazy Initialization:** El método `node(String name)` actúa como una fábrica de nodos. Verifica la existencia previa del nodo en el mapa; si no existe, lo instancia y lo registra, asegurando que no haya nodos huérfanos o duplicados.
- **Predict:** Cumpliendo con los requisitos del enunciado, el método `predict` se sobrecarga, permitiendo que el árbol procese clasificaciones tanto de colecciones de objetos (`T...` datos) como de la estructura `Dataset<T>`, delegando la evaluación de un único dato en un método privado `evaluar(T dato)`.

2. `Rama<T>`

La clase `Rama<T>` actúa como una arista direccional que conecta un nodo de origen con uno de destino, conteniendo la condición necesaria que se debe cumplir para poder pasar de un nodo a otro.

Se delega la lógica condicional a la interfaz funcional `Predicate<T>` de Java. Esto permite que el árbol sea completamente independiente a los datos que evalúa, definiendo las reglas de transición: "si *x* cumple *y*, ve por esta rama".

3. `Node<T>`

Cada instancia guarda su etiqueta, una colección de ramas salientes y un puntero referenciando al `DecisionTree` al que pertenece. Este último es necesario para que el nodo pueda validar reglas de unicidad en el `Set` del árbol padre antes de aceptar una nueva conexión y así no corromper la naturaleza de un árbol.

- **Camino por Defecto:** El nodo incorpora una referencia explícita a un `nodoDefecto`. Esto modela la condición *otherwise*, garantizando que el flujo de evaluación (evaluate) nunca se quede bloqueado.
- **Identidad Estricta:** Se han sobrescrito los métodos `equals()` y `hashCode()` basando la equivalencia del objeto en su etiqueta.

Apartado 3.

Con el propósito de independizar la clasificación de la estructura del árbol, se ha dotado a **DecisionTree<T>** de la capacidad de generar objetos **Predicate<T>** usando la interfaz **Predicate** de la librería **java.util.function**, permitiendo que una función evaluable que describe el camino recorrido.

Para implementar esta funcionalidad, se han aplicado las siguientes estrategias de diseño:

- **Recorrido en Profundidad (DFS):** El método `getPredicate(String label)` emplea un algoritmo recursivo que explora sistemáticamente las trayectorias desde la raíz. Esta búsqueda exhaustiva permite localizar el nodo que ostenta la etiqueta objetivo dentro de la jerarquía.
- **Encadenamiento Lógico de Reglas:** Durante el descenso, se utiliza la composición funcional mediante el método `.and()` de la interfaz **Predicate**. De este modo, se construye dinámicamente un conjunto con los criterios de cada rama del camino, definiendo las condiciones exactas para alcanzar el destino.
- **Mecanismo de Backtracking:** Si una ruta no desemboca en el nodo solicitado, el flujo de ejecución retrocede para descartar el predicado parcial y explorar nuevas alternativas. La recursividad solo finaliza satisfactoriamente cuando se valida la coincidencia con la etiqueta buscada.
- **Tratamiento de Etiquetas Inexistentes:** Ante la solicitud de un identificador ausente en el árbol, el sistema retorna *null*. Esto señala la inexistencia de una ruta lógica que satisfaga la clasificación requerida tras completar la inspección total de la estructura.

Apartado 4.

En este apartado expandiremos la clase decision Tree para que se puedan crear árboles de forma automática únicamente dado un conjunto de datos y el aprendizaje de sí mismo. Para ello hemos implementado la clase greedy Tree Learn que se encarga de aprender dado un árbol, para ello hemos tenido que implementar varios módulos distintos:

- **Interfaz LabelProvider<T, L>:** esta interfaz abstrae la lógica de clasificación. Su función principal consiste en asignar una etiqueta de tipo L a una entidad de tipo T, básicamente cada objeto T, tiene una etiqueta L.
- **Clase LabeledDataset<T, L>:** Hereda de la clase Dataset<T> para desempeñar el rol de repositorio de entrenamiento. Para poder ser instanciada por su constructor hace falta un IFeaturizer y un LabelProvider, facilitando así la obtención de la clasificación esperada para cada registro procesado.

Motor de Aprendizaje (GreedyTreeLearner<T, L>): Esta unidad encapsula la lógica necesaria para instanciar dinámicamente un DecisionTree<T> íntegro a partir de un LabeledDataset. La esencia del procedimiento reside en un mecanismo recursivo interno que implementa el pseudocódigo establecido mediante las siguientes determinaciones técnicas:

- **Caso Base y Nodos Hoja:** El algoritmo evalúa sistemáticamente las etiquetas del subconjunto de datos actual en cada iteración. Al identificar un conjunto puro, donde todos los registros comparten la misma etiqueta, la recursión finaliza. En esta etapa, la etiqueta se convierte a String mediante el método *.toString()* para asegurar la compatibilidad con los identificadores de nodo del árbol.
- **Estrategia de Splitting (División):** Cuando el conjunto de datos contiene etiquetas mezcladas, es necesario particionarlo. Para ello, se elige una característica al azar que servirá como criterio de división. Es vital que, una vez utilizada, esta característica se elimine de la lista de opciones disponibles para las siguientes llamadas recursivas. De no hacerlo, el algoritmo podría evaluar la misma característica repetidas veces, provocando un bucle infinito en la construcción del árbol.
- **Organización mediante Estructuras de Mapa:** Para efectuar la segregación de la información, se emplea un diccionario de tipo Map<Object, List<T>>. En este esquema, cada clave se corresponde con un valor distintivo de la característica evaluada, mientras que el valor asociado representa la sublista de elementos que cumplen dicha condición.
- **Construcción Dinámica de Conexiones:** Por cada subconjunto derivado, el sistema ejecuta una llamada recursiva para edificar el subárbol pertinente. Tras recuperar el nodo raíz resultante, el nodo de origen se vincula mediante una condición generada al vuelo. Esta transición se implementa instanciando un Predicate<T> que valida si la característica extraída del objeto coincide con el valor asignado a la rama.

Apartado 5.

El objetivo principal reside en optimizar la generación automatizada de la estructura jerárquica, permitiendo que el motor de aprendizaje (**GreedyTreeLearner**) seleccione de forma inteligente la característica más idónea para particionar los datos en cada etapa recursiva. Para garantizar un diseño modular y fácilmente ampliable, se ha integrado el **patrón de diseño Strategy**. Es decir creamos una interfaz y luego hacemos que distintas clases implementan dicha interfaz.

- **Interfaz FeatureSelectionStrategy<T, L>**: Actúa como el componente *Strategy*, definiendo un contrato para procesar los datos actuales y las características disponibles. Su fin último es retornar el nombre de la propiedad que minimice la dispersión de los datos.
- **Adaptación del Contexto**: La clase GreedyTreeLearner delega ahora la toma de decisiones a la estrategia seleccionada. Se mantiene la compatibilidad mediante la sobrecarga de constructores, ofreciendo una implementación aleatoria por defecto si no se especifica una heurística concreta.

Se han desarrollado dos estrategias específicas para dotar al sistema de criterios de decisión:

1. RandomStrategy<T, L> (Heurística base): Implementa una selección aleatoria como mecanismo de control. Para evitar efectos colaterales en las colecciones originales, utiliza una copia defensiva de la lista de características antes de aplicar un barajado aleatorio. Usamos el método de collections de shuffle y luego sacamos el primer elemento.

2. MisclassificationStrategy<T, L> (Heurística de Clasificación Errónea): Emplea una métrica analítica para reducir el error residual en cada nodo. El algoritmo evalúa cada característica siguiendo un proceso de cuatro fases:

- **Agrupación**: Realiza un particionado hipotético de la información basándose en los valores distintivos de la característica bajo análisis.
- **Cálculo de la etiqueta mayoritaria**: Identifica la moda estadística (clase predominante) en cada uno de los subconjuntos generados.
- **Contabilización de errores**: Evalúa el potencial de fallo restando la frecuencia de la etiqueta mayoritaria al volumen total del subgrupo.
- **Evaluación final**: Agrega las penalizaciones de todos los subgrupos y selecciona la característica con la puntuación de error mínima, produciendo árboles más compactos y eficientes.

Apartado 6.

El patrón se sustenta en dos interfaces principales que establecen el contrato de recorrido y procesamiento:

- **TreeElement:** Define el método `accept`. Como adaptación específica al árbol de decisión y la disposición jerárquica de sus elementos, este método exige el paso de la profundidad actual del elemento (`depth`) junto con el propio visitante como parámetros.
- **TreeVisitor:** Declara métodos de visita (`visitTreeNode` y `visitRama`). Esto garantiza que las clases concretas puedan aplicar lógicas específicas según el tipo de elemento que están procesando.

La responsabilidad de propagar el recorrido recae sobre los propios elementos del árbol, aplicando el mecanismo de *double dispatch*:

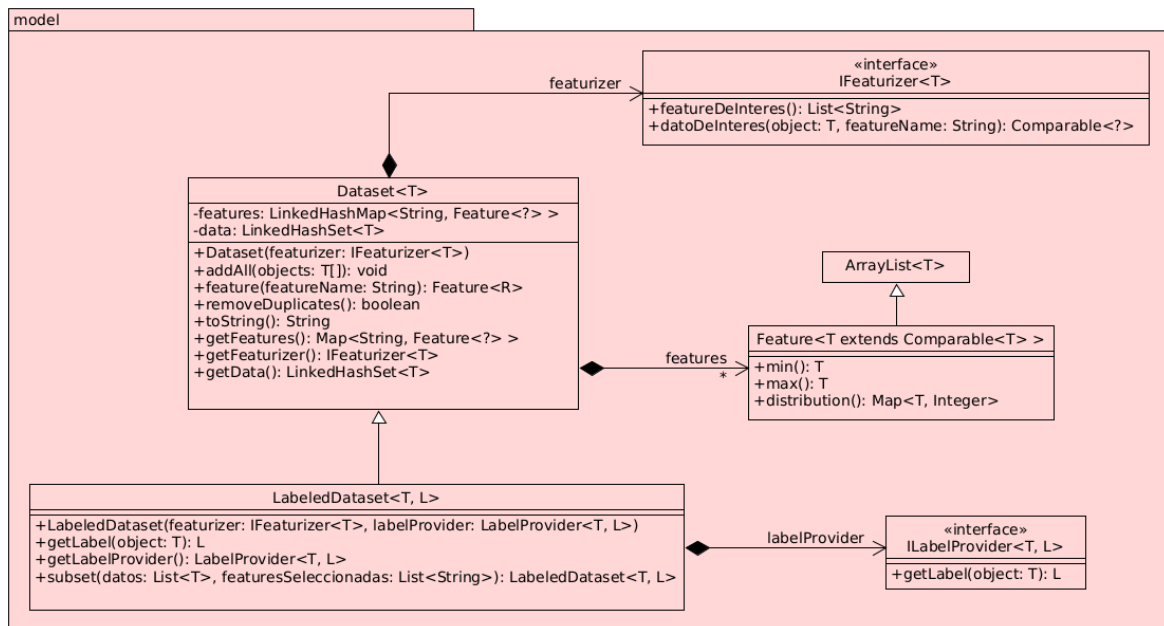
- **Nodos (Node):** El método `accept` visita primero el propio nodo y, acto seguido, itera sobre sus ramas propagando el visitante e incrementando la profundidad geométrica (`depth + 1`).
- **El camino otherwise:** Una decisión de diseño fundamental implementada en el nodo es el tratamiento del camino por defecto. Si existe un nodo `otherwise`, se instancia dinámicamente una Rama virtual o transitoria para envolverlo. Esto unifica la semántica del recorrido: a nivel del visitante, todo camino entre dos nodos es tratado uniformemente como una "Rama", simplificando drásticamente el código cliente que no requiere condiciones extra para casos especiales.
- **Ramas (Rama):** El método `accept` visita en primer lugar la rama, e inmediatamente después continúa llamando al método `accept` de su nodo destino.

Aprovechando esta base, se han desarrollado dos implementaciones que satisfacen diferentes necesidades operativas:

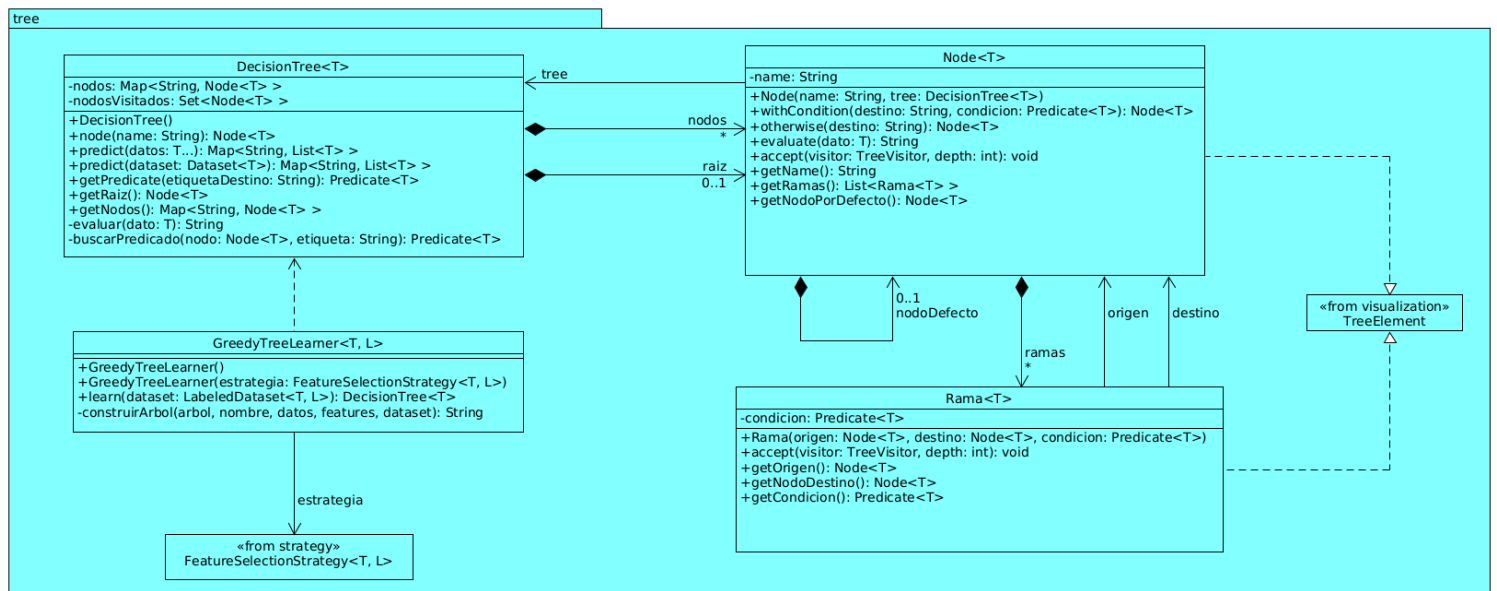
- **PlainTextTreeVisitor:** Genera una representación textual jerárquica. Esta clase aprovecha de forma directa la profundidad de cada elemento para calcular su sangría, y guarda el árbol en un `StringBuilder` que se puede recuperar con el método `getResult()`.
- **GraphvizTreeVisitor:** Actúa como un compilador que traduce la estructura del árbol a lenguaje DOT, interpretable por el software Graphviz. Acumula la declaración de grafos, atributos estéticos (`shape=box`), nodos direccionales y etiquetas descriptivas apoyándose en un `StringBuilder`. La visualización final se obtiene con el método `getResult()`, donde se devuelve la cadena completa.

Diagramas:

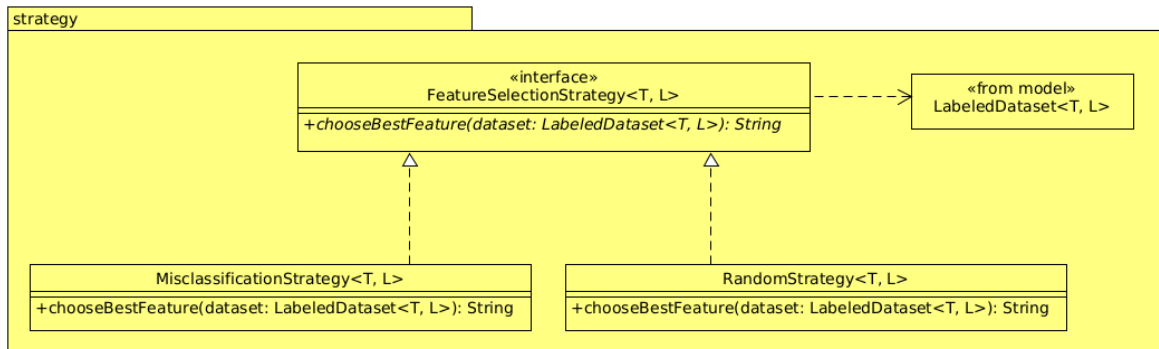
Paquete model.



Paquete tree.



Paquete **strategy**.



Paquete **visualization**.

